

Elder Health Monitoring & SOS Alert System (Python-Based)

1. Real-World Problem

Elderly individuals living alone often face medical emergencies without timely assistance. Lack of continuous monitoring increases risks and delays critical medical responses.

2. Objectives & Expected Outcomes

- Monitor simulated vitals in real-time.
- Detect abnormal heart rate and temperature.
- Generate SOS alerts for emergency situations.
- Provide a simple, automated monitoring solution.

Expected Outcome: A functioning Python program that simulates health monitoring and alerts upon detecting dangerous values.

3. Application of Course Concepts

The project applies core programming concepts such as:

- Conditionals and loops
- Modular coding practices
- Algorithm design
- Input/output operations
- Random data simulation

These concepts align with introductory programming and problem-solving courses.

4. Tools & Technologies Used

- Python 3.x
- Libraries: random, time

No backend, frontend, or hardware sensors are used.

5. Structured Development Process

A. Problem Definition

Elders require continuous monitoring for safety, but manual monitoring is unreliable. A simple automated system is needed.

B. Requirement Analysis

- Generate random vital signs.

- Compare with threshold values.
- Print warnings and SOS alerts.
- Continuous loop for real-time simulation.

C. Top-Down Design / Modularization

Main modules:

- generate_vitals()
- check_thresholds()
- display_output()
- trigger_sos()
- main_loop()

D. Algorithm Development

1. Generate random heart rate & temperature.
2. Display readings.
3. Compare values with safe limits.
4. If abnormal → SOS alert.
5. Repeat in loop with delay.

E. Implementation

Implementation done in Python using random value generation and conditional checks.

F. Testing & Refinement

- Tested using multiple value ranges.
- Adjusted thresholds.
- Ensured continuous program stability.
- Verified SOS alert triggers correctly.

6. Project Features

- Real-time simulated monitoring
- SOS alerts for critical readings
- Threshold-based health detection
- Console-based interface

7. System Workflow

1. Simulate data
2. Display readings
3. Check thresholds
4. Trigger SOS if needed

8. Project Structure

/src

main.py

/docs

Project_Report.pdf

/screenshots

recordings/

README.md

9. How to Run

1. Install Python 3
2. Navigate to /src directory
3. Run: python main.py

10. Future Improvements

- Real sensors integration
- Mobile app interface
- Cloud data logging
- Fall detection
- SMS/call-based SOS alerts

11. Author

Mohit Yadav